SEAH ZHANG JIAN

A24CS0297

Lab 3: Loops and Algorithms for Sequence Quality Control

Sequencing quality-control pipeline: design the rules on paper, then build qc.py with conditions and loops

Item	Detail
Mode	Individual
Weight	Formative (no marks in the current schedule)
Teaching week	Week 3 (26 Oct - 1 Nov 2026)
CLOs	CLO1
Competencies	C02, C11
AI level	Level 1: AI as tutor (PROPOSED policy). You may ask AI to explain concepts, give hints, explain an error message, or review code you wrote. You may NOT paste in the task and submit AI-written code or text as your answer.
AI log	Submit AI_USE_LOG.md (prompt log), or a signed nil return if you used no AI.

Overview

A sequencing quality-control pipeline decides, for every record, whether it is good enough to use. The rules are written down and agreed before any code is written: a record shorter than 500 bases is rejected; a record with more than 10 N bases, or with GC percentage outside 30 to 70 (both limits included), is sent for review; everything else passes. A second routine trims the low-quality tail from the 3' end of a read.

In this lab you first design the rules (pseudocode, a flowchart and a hand-trace table), then write three functions in qc.py and test every boundary. The lab is formative: it carries no marks in the course schedule, but it is checked by the same automatic marker as the marked labs. All data are synthetic.

Work in the starter repository. Every function has a name, a signature and a docstring. Replace the line `raise NotImplementedError` with your own code. Do not change the function names, the signatures or the default values.

What you will practise

- Write conditions with comparison operators, and/or/not, and see that the ORDER of rules changes the outcome.
- Test boundaries (499 and 500, 10 and 11 N, GC 30 and 70) and avoid the traps of text compared with numbers.
- Use for loops with counters and a while loop with a guard that cannot run past the start of a list.
- Design before coding: input, output, constraints, pseudocode, flowchart and a hand-trace table.
- Keep thresholds as named keyword-only parameters, not magic numbers.

Files and resources

- Starter repository: src/qc.py (three stubs), tests/test_visible.py, data/qc_records.csv (ten synthetic records), pytest.ini, requirements.txt, AI_USE_LOG.md.
- Run the tests with: python -m pytest. Check style with: ruff check src.

- Week 3 lecture notes: the rules in order, loops and accumulators, the while trim, pseudocode and flowcharts.

Instructions

- Do T1 (design) before you write any code. Save it as design/qc_design.md in your repository.
- Complete the three function tasks T2 to T4 in src/qc.py. Commit after each task with a clear message.
- Add at least three tests of your own in tests/test_lab03_own.py (two boundary tests and one trim test, see T5).
- Complete AI_USE_LOG.md. This lab is AI Level 1 (tutor): AI may explain and give hints but must not write your answers.

Tasks

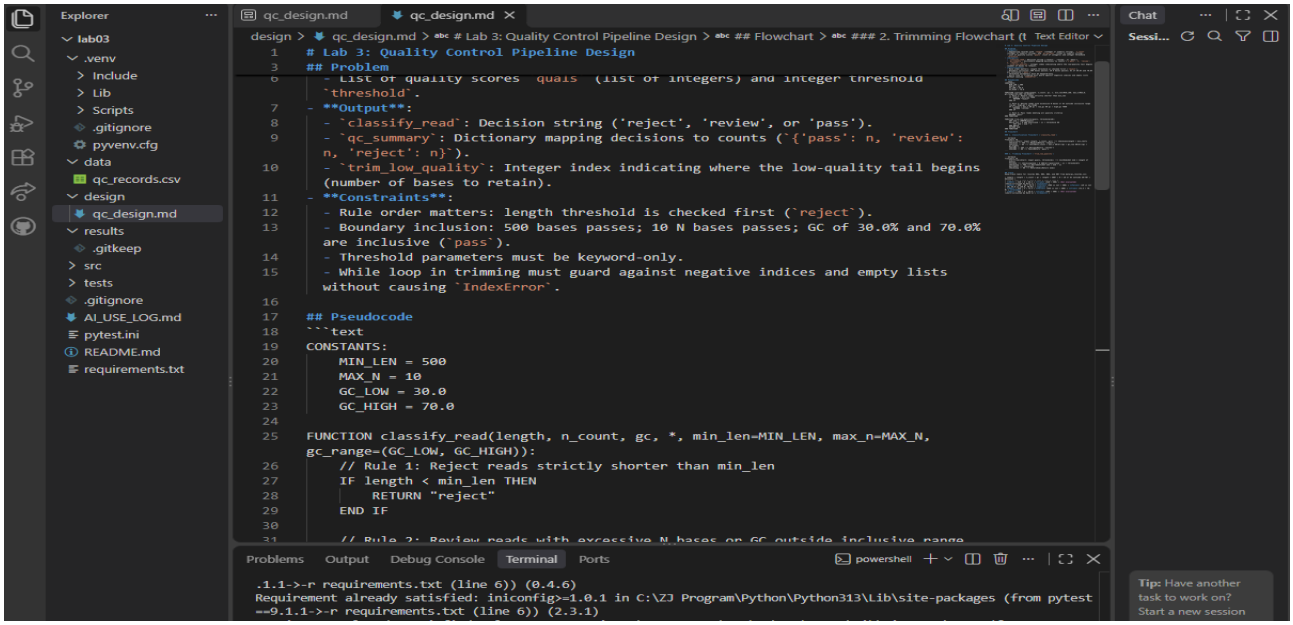
T1. Design first: pseudocode, flowchart and hand-trace table (3 marks)

Create design/qc_design.md with four headed sections: Problem (input, output, constraints), Pseudocode, Flowchart and Trace. The pseudocode covers classify_read AND the trim loop. The flowchart shows the rules in the order they are checked (a Mermaid code block, an ASCII diagram, or a link to a photo of a hand-drawn chart in design/). The trace is a Markdown table that you fill in BY HAND, before you run any code, for at least these four records of data/qc_records.csv: R03, R04, R05 and R07. Show each condition as its own column and give the decision.

```
# design/qc_design.md (skeleton)
## Problem
## Pseudocode
## Flowchart
## Trace
| sample | length | n_count | gc | length < 500? | N > 10 or GC outside 30-70? | decision |
|---|---|---|---|---|---|---|
```

Hint: Write the thresholds as names (MIN_LEN, MAX_N, GC_LOW, GC_HIGH) in the pseudocode. Decide in writing whether 500 bases and GC 70 pass, then make the code match your decision.

Answer:



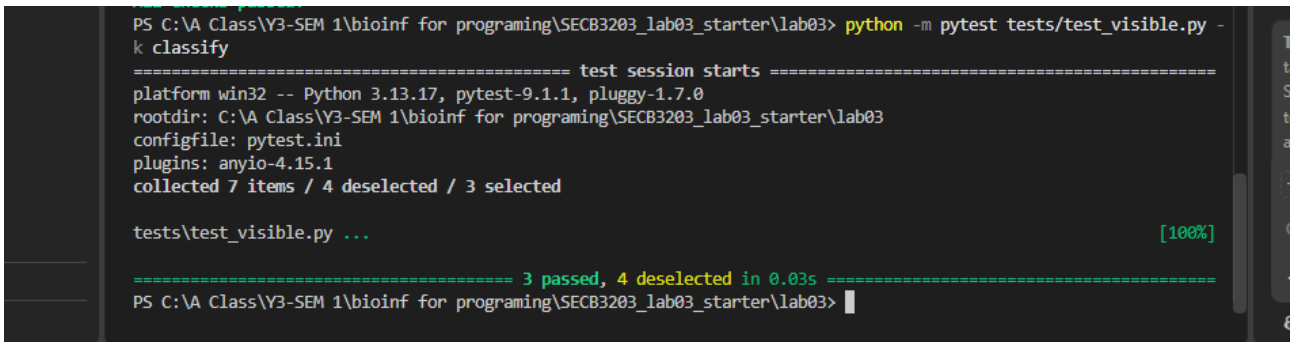
Quality control pipeline design document qc_design.md containing Problem description, Pseudocode, Flowcharts, and the trace table.

T2. classify_read (5 marks)

classify_read(length, n_count, gc, *, min_len=500, max_n=10, gc_range=(30.0, 70.0)) returns 'reject', 'review' or 'pass'. Rules, in this order: length below min_len gives 'reject'; otherwise more than max_n N bases, or GC outside the inclusive range gives 'review'; otherwise 'pass'. The three limits are keyword-only parameters. The function returns text and does not print.

Hint: A short read with bad GC is rejected, not reviewed: the order of your if statements is part of the specification.

Answer:



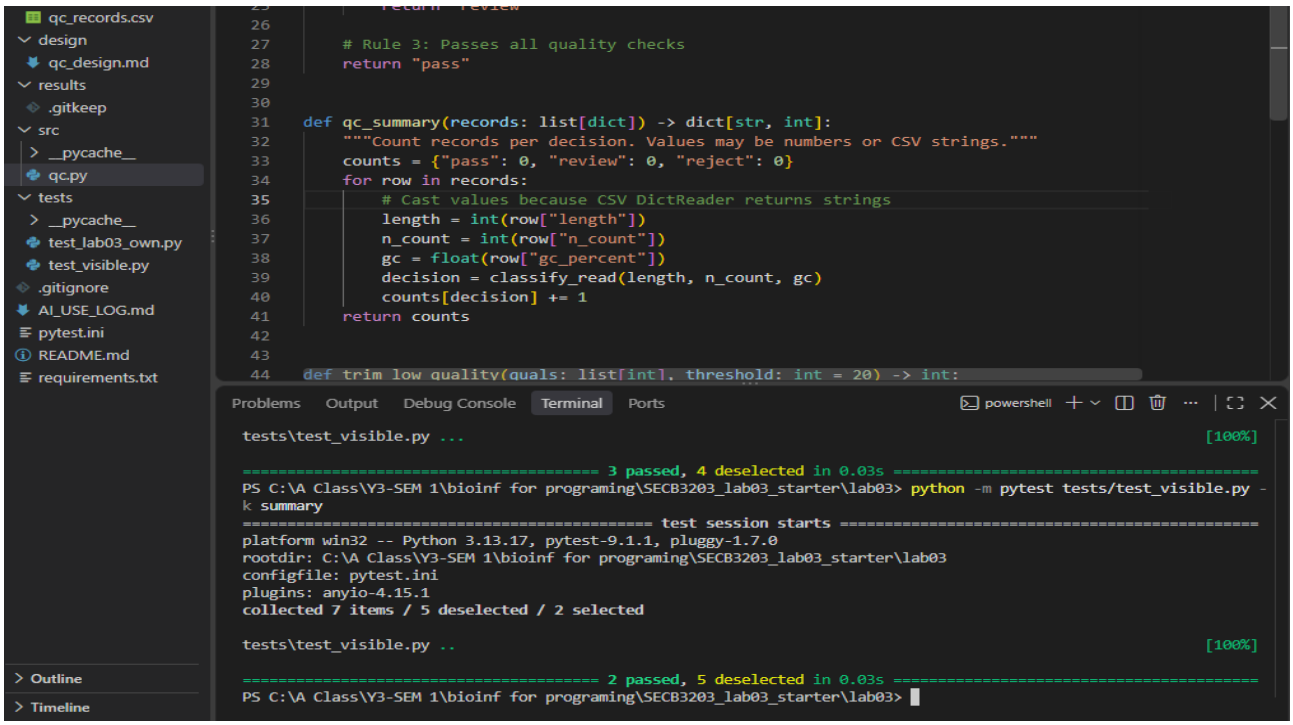
Implementation of classify read in src/qc.py with sequential quality control rules, verified by passing pytest boundary and visibility tests.

T3. qc_summary (3 marks)

qc_summary(records) takes a list of dictionaries with the keys length, n_count and gc_percent and returns how many records got each decision as {'pass': n, 'review': n, 'reject': n}. All three keys are always present, also when the count is 0 and when the list is empty. Values may be numbers or text read from a CSV file, so convert them before you compare. Do not change the records.

Hint: csv.DictReader gives every value as text: '500' < 499 raises TypeError. Use int() and float().

Answer:



Implementation of qc_summary in src/qc.py casting CSV string inputs to numerical types and aggregating classification counts, verified by pytest on mock rows and data/qc_records.csv.

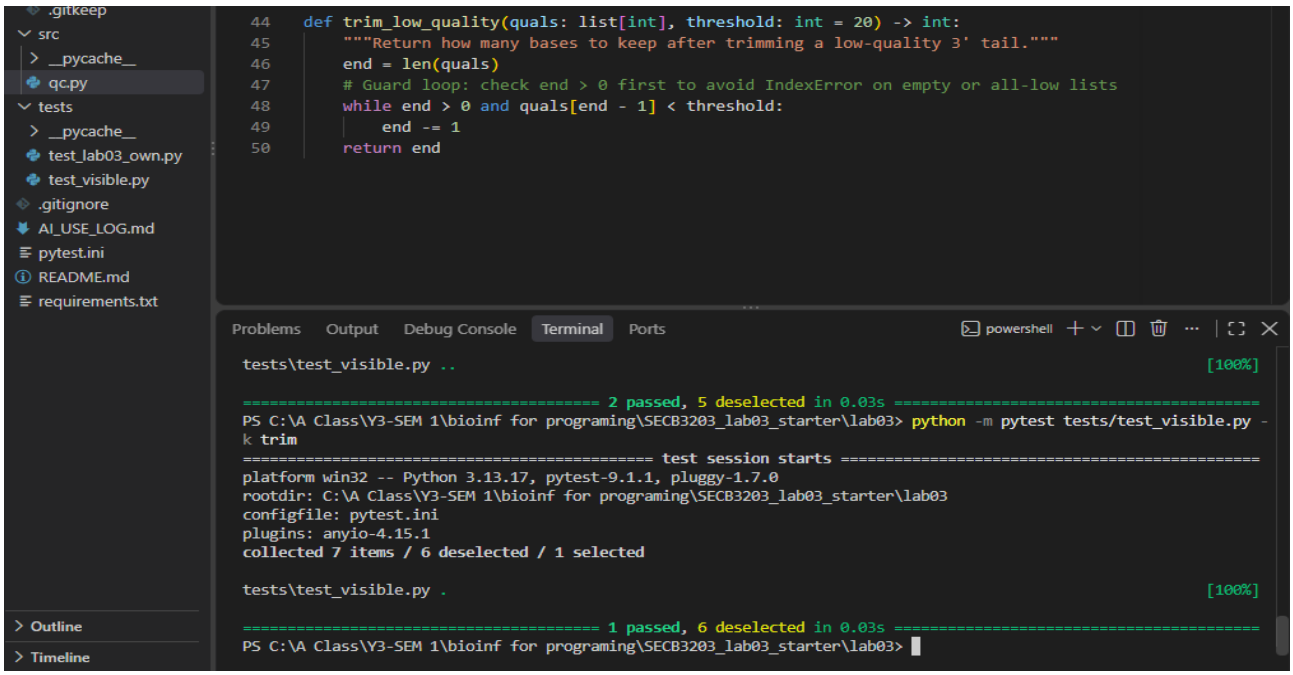
T4. trim_low_quality (3 marks)

trim_low_quality(quals, threshold=20) returns the index where the 3' low-quality tail starts, which is also the number of bases to keep. Start from the end and move left while the quality is BELOW the threshold. A score equal to the threshold is kept. Low scores that are not part of the tail stay. An empty list gives 0 and a list that is entirely low gives 0. Use a while loop. Do not change the list.

```
>>> trim_low_quality([30, 32, 35, 12, 10])
3
>>> trim_low_quality([5, 6, 7])
0
```

Hint: Test the bound first: end > 0 and quals[end - 1] < threshold. With no bound check an all-low list raises IndexError; with the check second, an empty list does.

Answer:



Implementation of trim_low_quality in src/qc.py using a guarded backward while-loop to identify the 3' low-quality boundary without IndexError, verified by pytest.

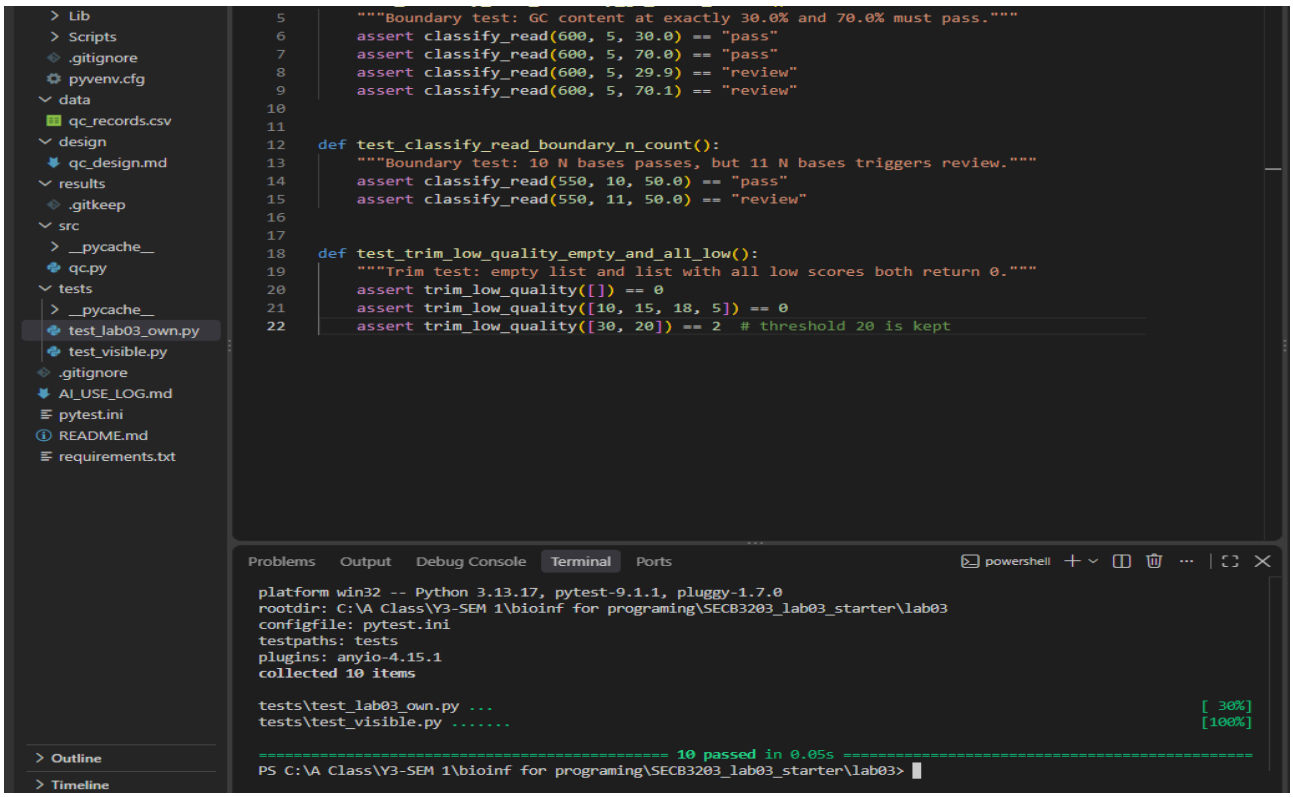
T5. Your own tests, clean code, AI log (4 marks)

Create tests/test_lab03_own.py with at least three tests of your own: two boundary tests for classify_read and one for trim_low_quality. Make `python -m pytest` pass, make `ruff check src` report no issues and complete AI_USE_LOG.md.

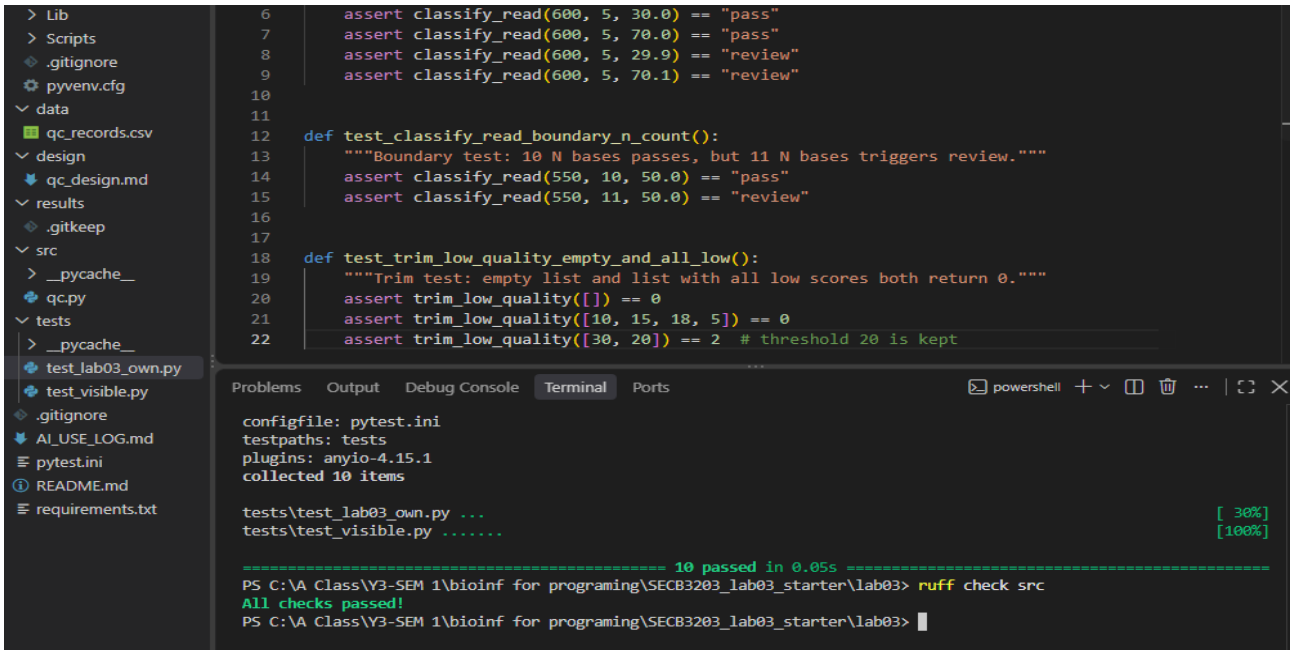
The rules at a glance

Order	Condition	Decision	Boundary to test
1	length < min_len (500)	reject	499 rejects, 500 does not
2	n_count > max_n (10), or GC < 30 or GC > 70	review	10 N is fine, 11 N is review; GC 30 and 70 are fine
3	none of the above	pass	-

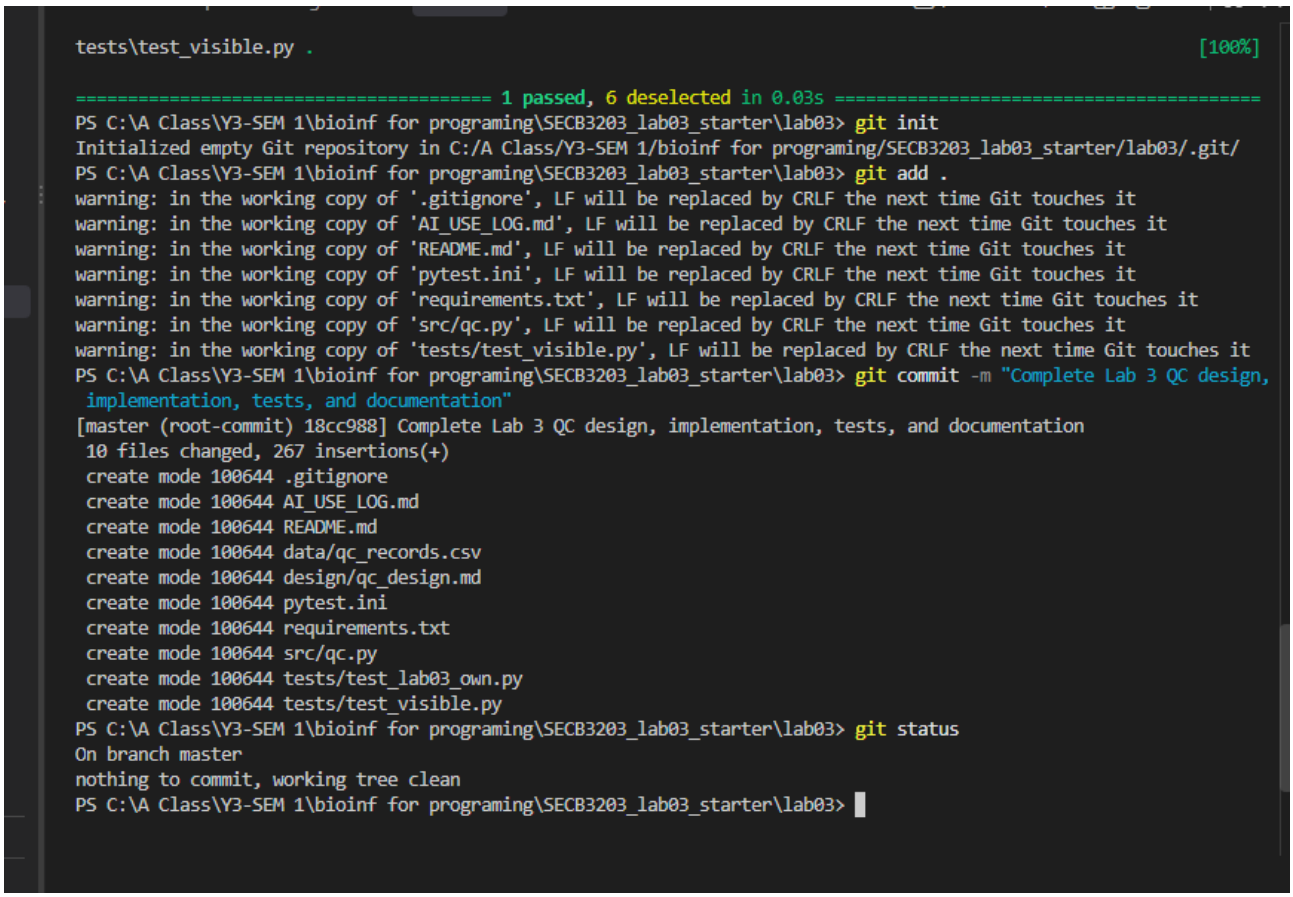
Answer:



Pytest test execution confirming all visible suite tests and custom boundary/trim tests pass successfully with zero failures.



Static analysis using Ruff confirming src/qc.py strictly complies with PEP 8 styling, naming conventions, and clean import rules.



Git status output verifying that all source code, unit tests, and design specifications are committed with a clean working tree.

What to submit

- Push your repository to GitHub (or submit the zip) before the lab deadline set in the course schedule.
- Include design/qc_design.md, src/qc.py, tests/test_lab03_own.py, AI_USE_LOG.md and README.md with the commands to run the tests.

How it is marked

#	Criterion	Marks	Marked by
R1	classify_read correct: rule order, 499/500, 10/11 N, GC 30/70 inclusive, keyword-only limits	5	Auto
R2	qc_summary correct: all keys present, text or numbers, empty list, input unchanged	3	Auto
R3	trim_low_quality correct: tail only, threshold kept, empty and all-low lists, no IndexError	3	Auto
R4	Own tests: tests/test_lab03_own.py has at least three tests and passes	1	Auto
R5	design/qc_design.md exists with Pseudocode, Flowchart and Trace sections, the three decisions, 500, a while loop and a trace table of at least three rows	1	Auto
R6	ruff reports no issues on src/	2	Auto
R7	AI use log submitted (prompt log or signed nil return) and valid	1	Auto
R8	Design quality: pseudocode, flowchart and hand-trace table are correct, complete and written before the code	2	Lecturer
R9	Individual Q&A: explains the rule order, one boundary test and how the while loop stops	2	Lecturer
	Total	20	

Automated checks use hidden tests as well as the visible ones, so passing the visible tests alone does not guarantee full marks. You must be able to explain your own code in the individual Q&A.